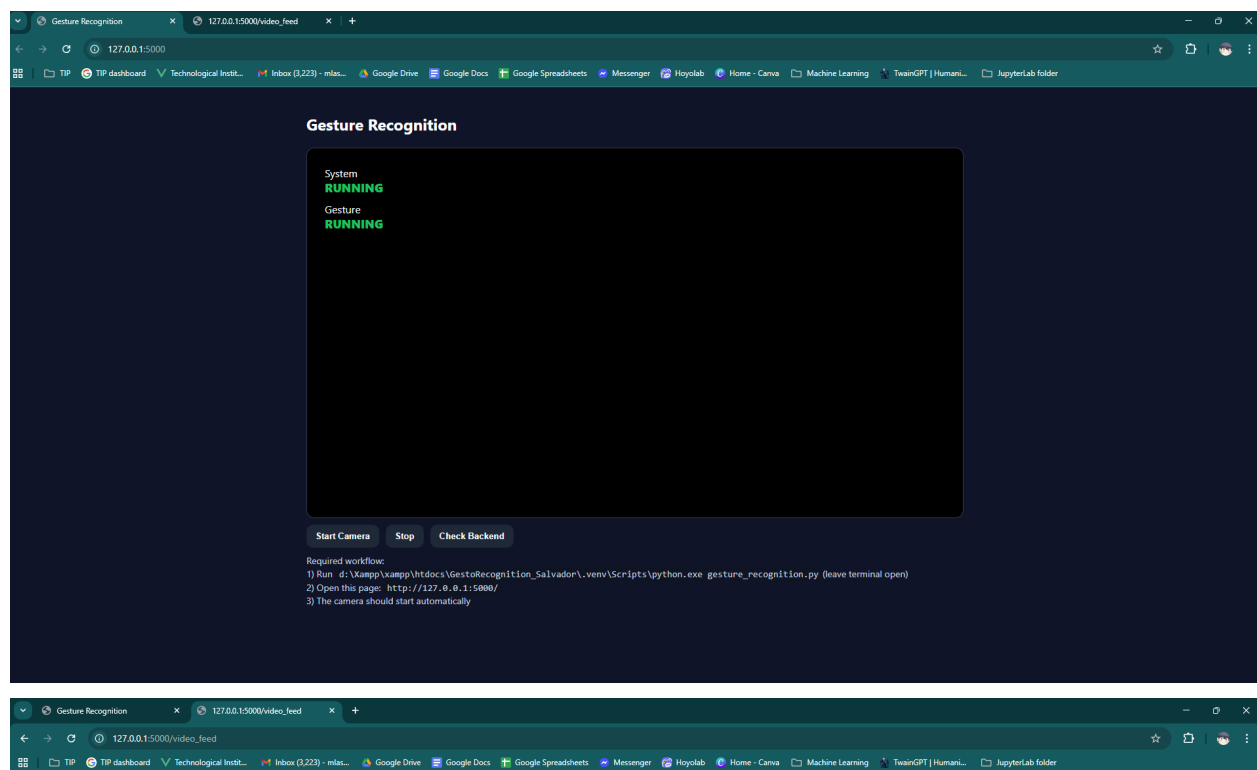




```
<?php ?>
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Gesture Recognition</title>

    <style>
      body {
        margin: 0;
        font-family: Segoe UI, Arial;
        background: #0f172a;
        color: #fff;
      }

      .container {
        max-width: 1000px;
        margin: auto;
        padding: 20px;
      }

      .stage {
        position: relative;
        aspect-ratio: 16/9;
        width: 100%;
        background: #000;
        border-radius: 12px;
        overflow: hidden;
        border: 1px solid #334155;
      }

      #stream{
        position: absolute;
        inset: 0;
        width: 100%;
        height: 100%;
        object-fit: cover;
      }
    </style>
  </head>
  <body>
    <div class="container">
      <div class="stage">
        <img alt="Stream" id="stream" />
      </div>
    </div>
  </body>
</html>
```

```
.hud {
  position: absolute;
  top: 15px;
  left: 15px;
  background: rgba(0,0,0, .6);
  padding: 12px;
  border-radius: 10px;
  min-width: 200px;
}
```

```
.status {
  font-weight: 900;
  font-size: 18px;
}
```

```
.good {
  color: #22c55e;
}
```

```
.warn{
  color: #f59e0b;
}
```

```
.bad {
  color: #ef4444;
}
```

```
.controls{
  margin-top: 10px;
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}
```

```
button {
  padding: 10px 14px;
  border-radius: 10px;
  border: none;
  font-weight: bold;
}
```

```

        cursor: pointer;
        background: #1e293b;
        color: #fff;
    }

    button:hover{
        background: #334155;
    }

    .note {
        margin-top: 10px;
        color: #cbd5e1;
        font-size: 14px;
        line-height: 1.4;
    }

    code {
        background: #111827;
        padding: 2px 6px;
        border-radius: 6px;
    }
</style>
</head>

<body>
    <div class="container">
        <h2>Gesture Recognition</h2>

        <div class="stage">
            <img id="stream" alt="Camera Stream" />
            <div class="hud">
                <div>System</div>
                <div id="sysText" class="status warn">Ready</div>
                <div style="margin-top:10px">Gesture</div>
                <div id="gestureText" class="status warn"> ---</div>
            </div>
        </div>
        <div class="controls">
            <button id="startBtn">Start Camera</button>
            <button id="stopBtn">Stop</button>
        </div>
    </div>

```

```

        <button id="checkBtn">Check Backend</button>
    </div>

    <div class="note">
        Required workflow:<br>
        1) Run
<code>d:\Xampp\xampp\htdocs\GestoRecognition_Salvador\.venv\Scripts\python
.exe gesture_recognition.py</code>(leave terminal open)<br>
        2) Open this page: <code>http://127.0.0.1:5000/</code><br>
        3) The camera should start automatically
    </div>
</div>

<script>
    //Change Python port, update here:
    const PY_BASE = "";
    const streamImg = document.getElementById("stream");
    const sysText = document.getElementById("sysText");
    const gestureText = document.getElementById("gestureText");

    let pollTimer = null;

    function setSystem(txt,type ="warn"){
        sysText.innerText = txt;
        sysText.className = "status " + type;
    }

    function setGesture(txt, type = "warn") {
        gestureText.innerText = txt;
        gestureText.className = "status " + type;
    }

    async function getJSON(url) {
        const res = await fetch(url, { cache: "no-store" });
        if (!res.ok) throw new Error("HTTP " + res.status);
        return res.json();
    }

    async function postJSON(url){
        const res = await fetch(url, { method: "POST" });

```

```

    if (!res.ok) throw new Error("HTTP " + res.status);
    return res.json();
  }

  async function checkBackend() {
    console.log("checkBackend called");
    try {
      const data = await getJSON(PY_BASE + "/health");
      setSystem(data.running ? "BACKEND OK (RUNNING)":
"BACKEND OK (IDLE)", "good");
      return true;
    } catch (e) {
      console.log("checkBackend failed:", e);
      setSystem("PYTHON BACKEND NOT RUNNING", "bad");
      return false;
    }
  }

  async function startSystem() {
    console.log("startSystem called");
    const ok = await checkBackend();
    if(!ok) {
      setGesture("---", "warn");
      streamImg.removeAttribute("src");
      return;
    }
    try {
      setSystem("STARTING ...", "warn");
      await postJSON(PY_BASE + "/start");

      //Load MJPEG Stream
      if (pollTimer) clearInterval(pollTimer);
      let frameCount = 0;
      pollTimer = setInterval(() => {
        streamImg.src = PY_BASE + "/video_feed?ts=" +
(frameCount++);

        }, 250);
      streamImg.src = PY_BASE + "/video_feed?ts=" +
(frameCount++);

      setSystem("RUNNING", "good");

```

```

        if(pollTimer) clearInterval(pollTimer);
        pollTimer = setInterval (async () => {
            try{
                const g = await getJSON(PY_BASE + "/gesture");
                const label =g.gesture || "---";
                const type =
                    (label === "UNKNOWN") ? "warn" :
                    (label === "NO HAND") ? "bad" :
                    (label === "STOPPED") ? "warn" : "good";
                setGesture(label,type);
            } catch (e){
                setSystem("BACKEND LOST", "bad");
                setGesture("---", "warn");
            }
        }, 200);

    } catch (e) {
        setSystem("FAILED TO START", "bad");
        setGesture("---", "warn");
    }
}

async function stopSystem(){
    try {await postJSON(PY_BASE + "/stop"); } catch (e) {}
    if (pollTimer) clearInterval(pollTimer);
    pollTimer = null;

    streamImg.removeAttribute("src");
    setSystem("STOPPED", "warn");
    setGesture("---", "warn");
}

document.getElementById("startBtn").onclick = startSystem;
document.getElementById("stopBtn").onclick = stopSystem;
document.getElementById("checkBtn").onclick =
checkBackend;

//Optional:Autocheck
(async () => {
    await checkBackend();

```

```
        startSystem();  
    }) ();  
    </script>  
    </body>  
</html>
```



```

import cv2
import math
import time
import numpy as np
from threading import Lock
from flask import Flask, Response, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

# MediaPipe initialization - handle both old and new API versions
mp_hands = None
mp_drawing = None
try:
    import mediapipe as mp
    mp_hands = mp.solutions.hands
    mp_drawing = mp.solutions.drawing_utils
except Exception:
    pass

# Force fallback for compatibility
mp_hands = None
mp_drawing = None

print("Script started, imports done")

lock = Lock()
cap = None
running = True
latest_label = "READY"

#--Helpers---
def dist(a,b):
    return math.hypot(a.x - b.x, a.y - b.y)

def finger_extended(hand,tip_id, pip_id, mcp_id):
    wrist = hand.landmark[0]
    tip = hand.landmark[tip_id]

```

```

    pip = hand.landmark[pip_id]
    mcp = hand.landmark[mcp_id]
    return dist(tip,wrist) > dist(pip, wrist) > dist(mcp,wrist)

def thumb_extended(hand):
    wrist = hand.landmark[0]
    thumb_tip = hand.landmark[4]
    thumb_ip = hand.landmark[3]
    return dist(thumb_tip,wrist) > dist(thumb_ip, wrist) * 1.05

def finger_folded(hand,tip_id, pip_id):
    wrist = hand.landmark[0]
    tip = hand.landmark[tip_id]
    pip = hand.landmark[pip_id]
    return dist(tip,wrist) < dist(pip, wrist)

def recognize_gesture(hand):
    thumb = thumb_extended(hand)
    index = finger_extended(hand, 8, 6, 5)
    middle = finger_extended(hand, 12, 10, 9)
    ring = finger_extended(hand, 16, 14, 13)
    pinky = finger_extended(hand, 20, 18, 17)

    index_fold = finger_folded(hand, 8, 6)
    middle_fold = finger_folded(hand, 12, 10)
    ring_fold = finger_folded(hand, 16, 14)
    pinky_fold = finger_folded(hand, 20, 18)

    if thumb and index and middle and ring and pinky:
        return "OPEN PALM"

    if (index_fold or not index) and (middle_fold or not middle) and
    (ring_fold or not ring) and (pinky_fold or not pinky) and (not thumb):
        return "CLOSED FIST"

    if index and middle and (ring_fold or not ring) and (pinky_fold or not
pinky):
        return "PEACE SIGN"

```

```

        if thumb and (index_fold or not index) and (middle_fold or not middle)
and (ring_fold or not ring) and (pinky_fold or not pinky):
            return "THUMBS UP"

    return "UNKNOWN"

#--Camera Control--
def ensure_camera_open():
    global cap
    if cap is None:
        cap = cv2.VideoCapture(0)
        cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
        print("Camera opened:", cap.isOpened())

def release_camera():
    global cap
    if cap is not None:
        cap.release()
        cap = None

def blank_frame(width=1280, height=720, text="STOPPED"):
    frame = np.zeros((height, width, 3), dtype=np.uint8)
    cv2.putText(frame, text, (40, 90),
                cv2.FONT_HERSHEY_SIMPLEX, 3, (0, 255, 0), 3)
    return frame

# -- Frame --
def get_frame():
    global latest_label, cap
    print("get_frame called")
    try:
        print("Attempting to ensure camera open")
        ensure_camera_open()
        print(f"Camera open: {cap is not None}, isOpened: {cap.isOpened()}
if cap else False}")

    if cap is None or not cap.isOpened():
        print("Camera not opened, returning error frame")

```

```

        frame = blank_frame(text="CAMERA NOT OPEN")
    else:
        ok, frame = cap.read()
        print(f"Frame read result: {ok}")
        if not ok or frame is None:
            print("Frame read failed, returning error frame")
            frame = blank_frame(text="CAMERA ERROR")
            latest_label = "CAMERA ERROR"
        else:
            print(f"Frame shape: {frame.shape}")
            frame = cv2.flip(frame, 1)
            label = "NO HAND"
            latest_label = label
            cv2.rectangle(frame, (10,10), (420,70), (0,0,0), -1)
            cv2.putText(frame, f"Gesture: {label}", (20, 55),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 255,
255), 2)

    ok, buf = cv2.imencode('.jpg', frame)
    print(f"Encode result: {ok}")
    if ok:
        print(f"Returning {len(buf.tobytes())} bytes")
        return buf.tobytes()
    else:
        print("Encode failed")
        return b''

except Exception as e:
    print(f"Exception in get_frame: {e}")
    import traceback
    traceback.print_exc()
    frame = blank_frame(text=f"ERROR: {str(e)[:20]}")
    ok, buf = cv2.imencode('.jpg', frame)
    return buf.tobytes() if ok else b''

if mp_hands is not None:
    with mp_hands.Hands(
        static_image_mode=False,
        max_num_hands=1,
        model_complexity=1,
        min_detection_confidence=0.6,
        min_tracking_confidence=0.6

```

```

        ) as hands:
            _generate_frames_with_hands(hands)
    else:
        _generate_frames_without_hands()

def _generate_frames_with_hands(hands):
    global latest_label, running
    while True:
        if not running:
            latest_label = "STOPPED"
            frame = blank_frame(text = "STOPPED - Click Start")
            ok, but = cv2.imencode(".jpg", frame)
            if ok:
                yield (b'--frame\r\n'
                       b'Content-Type: image/jpeg\r\n\r\n' + but.tobytes()
                       + b'\r\n')

                time.sleep(0.08)
                continue

        with lock:
            ensure_camera_open()
            ok, frame = cap.read()

        if not ok or frame is None:
            latest_label = "Camera Error"
            frame = blank_frame(text="CAMERA ERROR")
            ok2, buf2 = cv2.imencode(".jpg", frame)
            if ok2:
                yield (b'--frame\r\n'
                       b'Content-Type: image/jpeg\r\n\r\n' +
                       buf2.tobytes() + b'\r\n')

                time.sleep(0.1)
                continue

        frame = cv2.flip(frame, 1)
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = hands.process(rgb)

        label = "NO HAND"
        if results.multi_hand_landmarks:
            hand_landmarks = results.multi_hand_landmarks[0]

```

```

        label = recognize_gesture(hand_landmarks)
        mp_drawing.draw_landmarks(frame, hand_landmarks,
mp_hands.HAND_CONNECTIONS)

        latest_label = label

        #overlay label
        cv2.rectangle(frame, (10,10), (420,70), (0,0,0), -1)
        cv2.putText(frame, f"Gesture: {label}", (20, 55),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 255, 255), 2)

        ok3, buf3 = cv2.imencode(".jpg", frame)
        if not ok3:
            continue
        yield (b'--frame\r\n'
                b'Content-Type: image/jpeg\r\n\r\n' + buf3.tobytes() +
b'\r\n')

def _generate_frames_without_hands():
    print("Using fallback mode")
    global latest_label, running
    while True:
        if not running:
            latest_label = "STOPPED"
            frame = blank_frame(text = "STOPPED - Click Start")
            ok, but = cv2.imencode(".jpg", frame)
            if ok:
                yield (b'--frame\r\n'
                        b'Content-Type: image/jpeg\r\n\r\n' + but.tobytes()
+ b'\r\n')

                time.sleep(0.08)
                continue
        with lock:
            ensure_camera_open()
            ok, frame = cap.read()
            print("Frame read:", ok)
            if ok:
                print("Frame shape:", frame.shape)

        if not ok or frame is None:

```

```

        latest_label = "Camera Error"
        print("Camera read failed")
        frame = blank_frame(text="CAMERA ERROR")
        ok2, buf2 = cv2.imencode(".jpg", frame)
        if ok2:
            yield (b'--frame\r\n'
                   b'Content-Type: image/jpeg\r\n\r\n' +
buf2.tobytes() + b'\r\n')
            time.sleep(0.1)
            continue

        frame = cv2.flip(frame, 1)
        cv2.imwrite('frame.jpg', frame)

        label = "MediaPipe not available"
        latest_label = label

    #overlay label
    cv2.rectangle(frame, (10,10), (420,70), (0,0,0), -1)
    cv2.putText(frame, f"Gesture: {label}", (20, 55),
                  cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 255, 255), 2)

    ok3, buf3 = cv2.imencode(".jpg", frame)
    if not ok3:
        continue
    yield (b'--frame\r\n'
           b'Content-Type: image/jpeg\r\n\r\n' + buf3.tobytes() +
b'\r\n')

#--- API ---
@app.route("/health")
def health():
    return jsonify({"ok": True, "running": running})

@app.route("/start", methods=["POST"])
def start():
    global running, latest_label
    running = True

```

```

    latest_label = "RUNNING"
    return jsonify({"ok": True, "running": running})

@app.route("/stop", methods=["POST"])
def stop():
    global running, latest_label
    running = False
    latest_label = "STOPPED"
    with lock:
        release_camera()
    return jsonify({"ok": True, "running": running})

@app.get("/gesture")
def get_gesture():
    return jsonify({"gesture": latest_label, "running": running})

@app.route('/')
def index():
    with open('index.php', 'r') as f:
        return f.read()

@app.get("/video_feed")
def video_feed():
    try:
        frame_data = get_frame()
        if frame_data:
            return Response(frame_data, mimetype='image/jpeg')
        else:
            print("get_frame returned empty data")
            return Response(b'', status=500)
    except Exception as e:
        print(f"Error in video_feed: {e}")
        return Response(b'', status=500)

if __name__ == "__main__":
    print("Starting Flask server")
    app.run(host="127.0.0.1", port=5000, debug=False, threaded=True)

```